



**Politechnika
Śląska**

Modelowanie i analiza systemów informatycznych

dokumentacja projektu RSP - Rozproszony system plików

Piotr Skowroński, Krzysztof Czuba, grupa 2

17 grudnia 2025

Spis treści

1	Część 1.	2
1.1	Opis programu	2
1.2	Wymagania wstępne i środowisko	2
1.2.1	Wymagania systemowe i narzędzia	2
1.2.2	Wymagania sieciowe	2
1.3	Instalacja i Wdrożenie	2
1.4	Instrukcja Uruchomienia	3
1.4.1	Uruchomienie Backendu	3
1.4.2	Uruchomienie Frontendu	3

1 Część 1.

1.1 Opis programu

Implementacja rozproszonego systemu plików, z uwzględnieniem problemu zakleszczeń i algorytmu elekcji.

System składa się z części serwerowej w której działa jeden węzeł główny (**master**) oraz wiele węzłów jedynie przechowujących dane (**storage**), oraz z web UI umożliwiającego download/upload plików.

1.2 Wymagania wstępne i środowisko

1.2.1 Wymagania systemowe i narzędzia

Do uruchomienia systemu wymagane są następujące narzędzia:

- **System Operacyjny: Linux/MacOS** (testowane lokalnie).
- **Git** – pobranie repozytorium.
- **Docker i Docker Compose** – uruchomienie backendu (API, baza danych, klaster RSP).
- **Node.js (22.x)** i **pnpm** – uruchomienie frontendu.

1.2.2 Wymagania sieciowe

Aby klaster działał poprawnie, należy zapewnić komunikację TCP:

- Master powinien być osiągalny dla węzłów Storage na porcie 9000/TCP (lub innym skonfigurowanym).
- Na maszynie zewnętrznej, na której działa węzeł Storage, port tego węzła (np. 9004/TCP) musi zostać odblokowany w firewallu.
- Z Mastera powinno być możliwe zestawienie połączenia do **advertise-ip:port** węzła Storage.

1.3 Instalacja i Wdrożenie

Sekcja opisuje kroki niezbędne do uruchomienia systemu.

1. Pobranie repozytorium:

```
git clone https://github.com/H4wk507/distributed-file-system
cd distributed-file-system
```

2. Uruchomienie backendu (API + baza danych + klaster RSP):

```
docker compose up --build
```

Backend uruchomi automatycznie PostgreSQL, wykona migracje oraz wystartuje klaster RSP (master + 3 węzły storage).

1.4 Instrukcja Uruchomienia

1.4.1 Uruchomienie Backendu

Backend uruchamiany jest poprzez Docker Compose:

```
docker compose up --build
```

Komenda ta uruchomi:

- PostgreSQL (baza danych),
- API HTTP (<http://localhost:8080>),
- Klaster RSP: 1 master + 3 węzły storage.

1.4.2 Uruchomienie Frontendu

Frontend uruchamiany jest osobno (wymaga Node.js 22.x i pnpm):

```
cd frontend  
pnpm install  
pnpm dev
```

Po uruchomieniu interfejs WWW jest dostępny pod adresem <http://localhost:5173>.

Dodatkowe informacje

Piotr Skowroński - Architektura systemu, praca nad testami, implementacja kluczowych algorytmów, backend, frontend.

Krzysztof Czuba - backend, frontend, praca nad elementami przechowywania plików, dokumentacja.

Część II

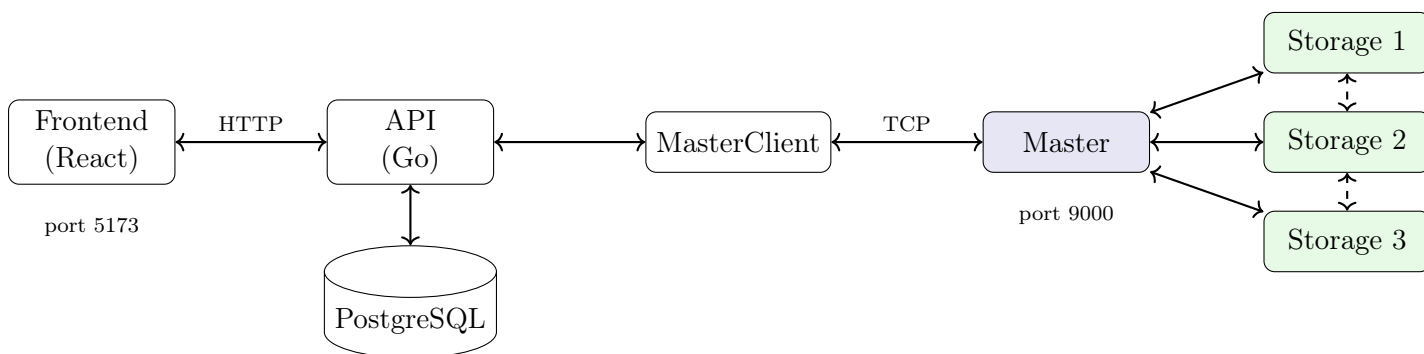
Opis działania

Poniższa sekcja opisuje praktyczną realizację mechanizmów rozproszonych w systemie, koncentrując się na sposobie rozwiązania kluczowych problemów: synchronizacji czasu, wzajemnego wykluczania, detekcji zakleszczeń oraz odporności na awarie. Cała logika rozproszona została zaimplementowana w katalogu `backend/dfs/`.

Architektura i komunikacja

System został podzielony na trzy logiczne warstwy, co zapewnia separację odpowiedzialności:

1. **Frontend** – interfejs użytkownika w technologii React. Komunikuje się wyłącznie z API.
2. **API (Go)** – warstwa pośrednia odpowiadająca za autoryzację (JWT) i przechowywanie metadanych w relacyjnej bazie danych. API nie przechowuje zawartości plików, lecz deleguje operacje I/O do klastra za pośrednictwem **MasterClient**.
3. **Klaster RSP** – warstwa składowania i logiki rozproszonej. Składa się z jednego węzła **Master** (koordynator) oraz wielu węzłów **Storage** (przechowywanie danych).



Rysunek 1: Architektura systemu RSP.

Węzły komunikują się dwoma kanałami:

- **JSON po TCP** – do wymiany komunikatów sterujących (elekcja lidera, żądania blokad, heartbeat).
- **Protokół binarny** – dedykowany do efektywnego transferu plików (uniknięcie narzutu base64/JSON).

Podczas uruchamiania nowy węzeł łączy się ze znanym adresem (seed node), wysyła komunikat powitalny (`MessageDiscovery`) i pobiera listę pozostałych węzłów w klastrze.

Synchronizacja czasu (Zegary Lamporta)

W systemach rozproszonych brak wspólnego zegara fizycznego utrudnia ustalenie kolejności zdarzeń. Aby rozwiązać ten problem, zastosowaliśmy **Zegary Lamporta**.

Mechanizm działa następująco: każdy węzeł utrzymuje lokalny licznik. Przy każdym zdarzeniu lokalnym lub wysłaniu wiadomości licznik jest inkrementowany. Przy odbiorze wiadomości węzeł aktualizuje swój zegar do wartości $\max(\text{lokalny}, \text{otrzymany}) + 1$.

Dzięki temu możemy określić relację "działo się przed" dla zdarzeń w systemie. W przypadku konfliktów (ta sama wartość zegara), jako kryterium rozstrzygające (tie-breaker) wykorzystujemy unikalne ID węzła. Pozwala to na deterministyczne sortowanie żądań dostępu do zasobów.

Wzajemne wykluczanie (Rozproszone blokady)

Dostęp do współdzielonych zasobów (plików) musi być synchronizowany, aby uniknąć niespójności. Zaimplementowaliśmy algorytm oparty na kolejkach żądań i wymianie komunikatów.

Procedura uzyskania blokady:

1. Węzeł rozsyła komunikat **LockRequest** do wszystkich pozostałych węzłów.
2. Każdy węzeł (w tym wnioskujący) dodaje żądanie do swojej lokalnej kolejki, sortowanej według czasu Lamporta.
3. Odbiorcy odsyłają potwierdzenie (**LockAck**).
4. Wnioskujący wchodzi do sekcji krytycznej tylko wtedy, gdy:
 - otrzymał potwierdzenia od **wszystkich** węzłów,
 - jego żądanie znajduje się na **szczybie** kolejki (ma najniższy znacznik czasu).
5. Po zakończeniu operacji węzeł zwalnia blokadę, a pozostali usuwają jego żądanie z kolejki.

Wymóg uzyskania potwierdzeń od całej grupy gwarantuje spójność widoku kolejki.

Wykrywanie zakleszczeń (Graf oczekiwania)

System blokad jest podatny na zakleszczenia (deadlock), np. gdy węzeł A czeka na zasób zajęty przez B, a B czeka na zasób zajęty przez A.

Rolę arbitra pełni tutaj **Master**:

- Utrzymuje globalny **graf oczekiwania** (wait-for graph), odwzorowujący zależności między węzłami.
- Cyklicznie (co 10s) analizuje graf w poszukiwaniu cykli.
- Wykrycie cyklu oznacza zakleszczenie. Master wybiera węzeł o najniższym priorytecie w cyklu ("ofiara") i wysyła mu polecenie **LockAbort**.
- Węzeł-ofiara musi zwolnić blokady, co przerywa cykl i umożliwia postęp pozostałym procesom.

Awaria i rekonfiguracja (Algorytm Bully)

System musi być odporny na awarię koordynatora (Mastera). W takim przypadku pozostałe węzły muszą automatycznie wyłonić nowego lidera. Zastosowaliśmy algorytm **Bully**.

1. Węzły monitorują komunikaty **Heartbeat** od Mastera. Brak sygnału przez 15s jest traktowany jako awaria.
2. Węzeł wykrywający awarię rozpoczyna elekcję, wysyłając zapytanie do wszystkich węzłów o wyższym identyfikatorze (priorytecie).
3. Jeśli węzeł o wyższym priorytecie odpowie, przejmuje proces elekcji.
4. Jeśli żaden węzeł o wyższym priorytecie nie odpowie, inicjator ogłasza się nowym Masterem.

Po objęciu funkcji nowy Master przeprowadza synchronizację metadanych, zbierając raporty o posiadanych plikach od wszystkich węzłów Storage, co przywraca spójność systemu.

Dystrybucja danych (Consistent Hashing)

Aby zapewnić równomierne obciążenie i minimalizować konieczność przenoszenia danych przy zmianach w klastrze, zastosowaliśmy **Consistent Hashing**.

- Przestrzeń haszy traktowana jest jako pierścień.
- Każdy węzeł fizyczny jest mapowany na wiele punktów na pierścieniu (węzły wirtualne), co poprawia rozkład danych.
- Plik jest przypisywany do węzła, którego pozycja na pierścieniu jest pierwszą napotkaną zgodnie z ruchem wskazówek zegara od hasha pliku.

Dodatkowo system utrzymuje **współczynnik replikacji** $r = 3$. Plik jest replikowany na trzy kolejne węzły fizyczne na pierścieniu, co zabezpiecza dane przed awarią pojedynczych maszyn.

Warstwa składowania (Storage)

Węzły przechowują pliki w systemie **Content-Addressed Storage**, gdzie identyfikatorem pliku jest hash jego zawartości (SHA256).

- Ścieżka pliku: `data/ab/abcd123...dat`
- Metadane: `data/ab/abcd123...metadata.json`

Podejście to zapewnia natywną **deduplikację** – identyczne pliki przesłane przez różnych użytkowników są przechowywane fizycznie tylko raz, co oszczędza przestrzeń dyskową.

Transfer plików (Streaming)

Ze względu na ograniczenia wydajnościowe formatu JSON przy przesyłaniu dużych porcji danych binarnych, zaimplementowaliśmy dedykowany protokół oparty na czystym TCP.

Struktura pakietu przesyłanego w protokole strumieniowym (łącznie 64 bajty nagłówka):

- **Magic Byte** (1 bajt): Identyfikator operacji (0x01 - Upload, 0x02 - Download).
- **Session ID** (36 bajtów): Unikalny identyfikator sesji (UUID) w postaci ciągu znaków.
- **File Size** (8 bajtów): Rozmiar przesyłanego pliku (int64, Big Endian).
- **Chunk Size** (4 bajty): Rozmiar pojedynczego fragmentu danych (int32, Big Endian).
- **Reserved** (15 bajtów): Pole rezerwowe (padding) dla wyrównania do 64 bajtów.
- **Data Stream**: Ciągły strumień bajtów pliku (payload) o długości **File Size**.

Proces przesyłania odbywa się z pominięciem Mastera: klient otrzymuje od Mastera listę adresów węzłów Storage, a następnie nawiązuje z nimi bezpośrednie połączenia równoległe. Eliminuje to wąskie gardło na węźle koordynującym.

Algorytmy

Poniżej przedstawiono pseudokod kluczowych algorytmów zastosowanych w systemie.

Aktualizacja zegara Lamporta

Data: Lokalny zegar L_i , (opcjonalnie) odebrany znacznik czasu t
Result: Nowa wartość L_i
if *zdarzenie lokalne lub wysłanie wiadomości* **then**
 $L_i \leftarrow L_i + 1$;
else
 $L_i \leftarrow \max(L_i, t) + 1$;
end

Algorithm 1: Zegar logiczny Lamporta.

Rozproszona blokada (kolejka Lamporta + ACK)

Data: Zasób resourceID, zbiór znanych węzłów peers
Result: Wejście do sekcji krytycznej lub oczekiwanie
RequestLock(resourceID);
 $L_i \leftarrow L_i + 1$;
Utwórz LockRequest(resourceID, nodeID=i, logicalTime= L_i);
Dodaj żądanie do lockQueue[resourceID] i posortuj po (*logicalTime, nodeID*);
Zainicjalizuj pendingAcks[resourceID];
Rozgłoś LockRequest do wszystkich peers;
OnReceiveLockRequest(req);
Dodaj req do lokalnej kolejki i posortuj;
Jeśli lockHolders[resourceID] istnieje i $\neq req.nodeID$, dodaj krawędź
 $req.nodeID \rightarrow holder$ do grafu oczekiwania;
Odeślij LockAck(resourceID) do nadawcy;
CanEnterCriticalSection(resourceID);
if *moje żądanie jest pierwsze w kolejce i liczba ACK $\geq |peers|$* **then**
 return true;
end
return false;
EnterCriticalSection(resourceID);
if *CanEnterCriticalSection(resourceID) == true* **then**
 Rozgłoś LockAcquired(resourceID);
end

Algorithm 2: Rozproszona blokada zasobu.

Wykrywanie i usuwanie zakleszczeń (graf oczekiwań)

Data: Graf oczekiwań $G = (V, E)$, priorytety węzłów
Result: Przerwanie cyklu (jeśli wykryty)
if *węzeł nie jest master* **then**
 | **return**
end
if $HasCycle(G)$ zwraca *true* oraz ścieżkę cyklu C **then**
 | $victim \leftarrow \arg \min_{v \in C} priority(v)$;
 | Wyślij LockAbort do $victim$;
end

Algorithm 3: Detekcja cyklu i wybór ofiary (master).

Elekcja lidera Bully

Data: Priorytet własny p_i , lista **peers**
Result: Ustalenie roli **master**
Wyślij Election do wszystkich węzłów o priorytecie $> p_i$;
if *nie istnieje żaden węzeł o priorytecie $> p_i$* **then**
 | Rozgłoś Coordinator i oczekuj krótko na ewentualny Coordinator od wyższego priorytetu;
 | Ustaw rolę **master**;
 | **return**
end
if *otrzymano OK w czasie electionTimeout* **then**
 | Zakończ elekcję (wyższy priorytet przejmie rolę);
 | **return**
end
Rozgłoś Coordinator i ustaw rolę **master**;

Algorithm 4: Algorytm elekcji lidera Bully.

Upload pliku (master + streaming do storage)

Data: FileID, Filename, ContentType, Size
Result: Zapis pliku na $r = 3$ storage nodes i aktualizacja metadanych
Master::
Wybierz r węzłów **storage** na podstawie consistent hashing (pierścień);
Utwórz **sessionID** i prześlij do wybranych węzłów **StreamUploadReady(sessionID, FileID, ...)**;
Zwróć klientowi listę adresów strumieniowania (**ip:(port+100)**);
Storage::
Po odebraniu strumienia zweryfikuj, czy **sessionID** istnieje;
Odczytaj dokładnie **Size** bajtów i zapisz w **LocalStorage**, licząc SHA256;
Wyślij do mastera **StreamStoreAck(sessionID, hash, nodeID)**;

Algorithm 5: Przepływ uploadu przez protokół streaming.

Bazy danych

System wykorzystuje relacyjną bazę danych PostgreSQL do przechowywania informacji o użytkownikach oraz metadanych plików. Sama zawartość plików przechowywana jest w klastrze RSP (węzły `storage`), natomiast baza przechowuje jedynie metadane i stan replik.

Typy i tabele

Schemat jest definiowany w migracji `backend/migrations/000001_init.up.sql`.

- `users` – użytkownicy aplikacji:
 - `id` (UUID, PK), `username` (UNIQUE), `email` (UNIQUE), `password_hash`, `role` (ENUM `user_role`), `created_at`, `updated_at`.
- `files` – metadane plików (nie dane binarne):
 - `id` (UUID, PK), `name`, `size` (BIGINT), `hash`, `content_type`, `owner_id` (FK → `users.id`), `created_at`, `updated_at`.
- `replicas` – informacja o replikach danego pliku w klastrze RSP:
 - `id` (UUID, PK), `file_id` (FK → `files.id`), `node_id` (UUID węzła RSP), `status` (ENUM `replica_status`), `created_at`, `updated_at`.
 - Constraint: `UNIQUE(file_id, node_id)` – zapobiega duplikatom replik na tym samym węźle.

Dodatkowo zdefiniowane są typy ENUM:

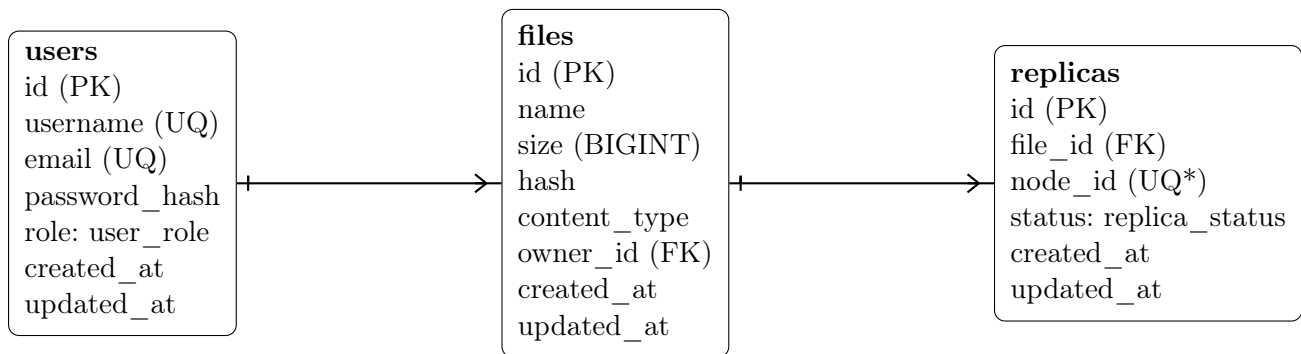
- `user_role` {`user`, `admin`},
- `replica_status` {`syncned`, `syncing`, `failed`}.

W każdej tabeli pole `updated_at` jest automatycznie aktualizowane przez trigger `update_updated_at_`.

Relacje i integralność

- Relacja `users` (1) -- (N) `files`: jeden użytkownik może posiadać wiele plików.
- Relacja `files` (1) -- (N) `replicas`: jeden plik może mieć wiele replik na węzłach RSP.
- Klucze obce mają `ON DELETE CASCADE` oraz `ON UPDATE CASCADE` (usunięcie użytkownika usuwa jego pliki; usunięcie pliku usuwa rekordy replik).

Diagram ERD



Rysunek 2: Diagram ERD bazy danych. *UQ = część klucza unikalnego (file_id, node_id).

Implementacja systemu

Sekcja opisuje implementację systemu z perspektywy podziału na moduły, ich odpowiedzialności oraz kluczowych przepływów (RSP i API/UI).

Struktura projektu

- `backend/` – implementacja RSP (węzły) oraz API (Go).
- `frontend/` – interfejs WWW (React + Vite).
- `backend/migrations/` – migracje bazy danych (PostgreSQL).
- `docker-compose.yml` – uruchamianie środowiska i klastra.

Backend API (Go)

Punkt wejścia API znajduje się w `backend/cmd/api/main.go`. Warstwa API jest zorganizowana typowo dla aplikacji usługowej:

- `backend/internal/handlers/` – kontrolery HTTP (m.in. `auth_handler.go`, `file_handler.go`, `metrics_handler.go`).
- `backend/internal/services/` – logika domenowa (np. autoryzacja, obsługa plików).
- `backend/internal/database/` – połączenie z PostgreSQL.
- `backend/internal/models/` – modele danych: `User`, `File`, `Replica`.
- `backend/internal/middleware/` – middleware (np. autoryzacja JWT).

API pełni rolę warstwy użytkowej: rejestracja/logowanie oraz operacje na plikach (upload/download/delete) delegowane do klastra RSP.

Klient Master (MasterClient)

Komunikacja między warstwą API a klastrem RSP odbywa się przez `MasterClient` (`backend/dfs/client`). Jest to klient TCP, który:

- utrzymuje połączenie z aktualnym masterem klastra,
- automatycznie odkrywa nowego mastera po awarii (failover) poprzez odpytanie seed nodes,
- udostępnia metody do operacji na plikach:
 - `InitStreamUpload` / `WriteChunk` / `Finalize` – upload w trybie chunked (dla dużych plików),
 - `UploadFileStream` – upload całego pliku w jednym wywołaniu,
 - `DownloadFileStream` – download z weryfikacją hash SHA256,
 - `DeleteFile` – usunięcie pliku z klastra.

Przepływ uploadu przez `MasterClient`:

1. API wywołuje `InitStreamUpload` – master zwraca `sessionID` oraz listę adresów storage nodes.
2. `MasterClient` otwiera połączenia TCP do wszystkich storage nodes i wysyła nagłówek binarny.
3. API streamuje dane przez `WriteChunk` – dane są równolegle wysyłane do wszystkich replik.
4. `Finalize` czeka na potwierdzenia od storage nodes i zwraca hash pliku.

Węzły RSP (Go)

Punkt wejścia węzła RSP znajduje się w `backend/cmd/node/main.go`. Rdzeń stanowi struktura `Node` w `backend/dfs/node/node.go`, która uruchamia:

- serwer TCP do komunikacji sterującej (wiadomości JSON),
- serwer strumieniowania (port `port+100`) do transferu danych plików,
- pętle monitorujące: heartbeat, zdrowie peerów, time-outy blokad, detekcję zakleszczeń.

Kluczowe moduły:

- **Protokół wiadomości.** Typy wiadomości i struktury danych są w `backend/dfs/common/types.go` (m.in. `MessageWithTime`, `LockRequest`). Każda wiadomość przenosi znacznik czasu Lamporta, a odbiorca aktualizuje zegar regułą $L_i := \max(L_i, t) + 1$.
- **Elekcja lidera.** Moduł `backend/dfs/election/bully.go` implementuje Bully; po wykryciu braku heartbeat lidera uruchamiana jest elekcja, a po wygranej ustawiana rola `master` oraz wykonywana synchronizacja metadanych.

- **Przechowywanie danych.** `backend/dfs/storage/local_storage.go` zapisuje pliki adresowane zawartością (SHA256) oraz metadane w plikach JSON, utrzymując indeks `index.json`. Dzięki temu identyczna zawartość mapuje się na ten sam hash.
- **Rozmieszczenie i replikacja.** `backend/dfs/hashing/hash.go` realizuje consistent hashing z węzłami wirtualnymi (domyślnie 150). Master wybiera $r = 3$ węzły storage jako repliki, a po elekcji wykonuje `MetadataRequest/Response` i naprawia brakujące repliki przez `ReplicateFile`.
- **Zakleszczenia i blokady.** Blokady realizowane są przez kolejkę żądań sortowaną po (L, id) i mechanizm ACK. Master buduje graf oczekiwania (wait-for graph) i wykrywa cykle (DFS), a następnie wysyła `LockAbort` do ofiary o najniższym priorytecie w cyklu.
- **Streaming.** `backend/dfs/streaming/` zawiera binarny protokół transferu (nagłówek o stałej długości + stream danych) oraz zarządzanie sesjami uploadu. Inicjalizacja sesji odbywa się u mastera, a klient wysyła dane bezpośrednio do wybranych storage nodes.

Frontend (React + Vite)

Frontend znajduje się w `frontend/`.

- `src/pages/` – widoki (np. logowanie, monitoring, węzły, pliki).
- `src/hooks/` – logika komunikacji z API (np. pobieranie listy plików, upload/download).
- `src/components/` – komponenty UI (lista plików, podgląd, upload, widok węzłów).

UI wykorzystuje API do operacji użytkownika, a sam transfer danych plików przebiega zgodnie z protokołem streaming (init u mastera + upload/download do storage).

Przykładowe fragmenty kodu

Poniżej przedstawiono fragmenty implementacji ilustrujące kluczowe mechanizmy rozproszone.

Warunek wejścia do sekcji krytycznej (`node/node.go`) – węzeł może wejść tylko gdy jest pierwszy w kolejce i zebrał wszystkie ACK:

```

1 func (n *Node) CanEnterCriticalSection(resourceID string) bool {
2     // ... timeout handling ...
3     first := n.lockQueue[resourceID][0]
4     if first.NodeID != n.ID {
5         return false // nie jesteśmy pierwsi w kolejce
6     }
7     expectedAcks := len(n.peers)
8     receivedAcks := 0
9     for _, acked := range n.pendingAcks[resourceID] {
10         if acked { receivedAcks++ }
11     }
12     return receivedAcks >= expectedAcks
13 }

```

Porównywanie żądań blokad (`common/lamport.go`) – sortowanie żądań po czasie Lamporta z tie-breakiem na ID węzła:

```
1 func CompareLockRequests(a, b *LockRequest) bool {
2     if a.LogicalTime != b.LogicalTime {
3         return a.LogicalTime < b.LogicalTime
4     }
5     return a.NodeID.String() < b.NodeID.String()
6 }
```

Reakcja na elekcję Bully (`election/bully.go`) – węzeł o wyższym priorytecie odpowiada OK i sam startuje elekcję:

```
1 func (b *BullyElector) handleElection(ctx context.Context, msg Message)
   error {
2     var nodeInfo common.NodeInfo
3     json.Unmarshal(msg.Payload, &nodeInfo)
4
5     if b.node.GetPriority() > nodeInfo.Priority {
6         // Mamy wyższy priorytet - odpowiadamy OK
7         msgOk := Message{Type: MessageOK, From: b.node.GetID()}
8         go b.node.SendMessage(nodeInfo.IP, nodeInfo.Port, msgOk)
9         b.electionMutex.Lock()
10        inProgress := b.electionInProgress
11        b.electionMutex.Unlock()
12        // Sami startujemy elekcję (może wygramy)
13        if !b.electionInProgress && b.node.GetRole() != RoleMaster {
14            go b.StartElection(ctx)
15        }
16    }
17    return nil
18 }
```

Testy

Testy jednostkowe zaimplementowano w Go w katalogu `backend/dfs/` (pliki `_test.go`). Pokrywają one głównie logikę algorytmiczną (bez uruchamiania pełnego klastra).

Uruchamianie

`make test`

Zakres

- `backend/dfs/common/waitfor-graph_test.go` – testy detekcji cyklu w grafie oczekiwania (przypadki: graf pusty, pętla własna, cykle 2- i 3-węzłowe, graf rozłączny).
- `backend/dfs/hashing/hash_test.go` – testy consistent hashing (deterministyczność HashKey, węzły wirtualne, dodawanie/usuwanie węzłów, stabilność wyboru replik dla tego samego FileID).

- `backend/dfs/storage/local_storage_test.go` – testy `LocalStorage` (`SaveFile/GetFile/Delete` idempotencja usuwania, zapis/odczyt `index.json`, deduplikacja: ta sama zawartość $\Rightarrow$ ten sam hash).
- `backend/dfs/node/node_test.go` – testy logiki węzła (kolejki/ACK blokad, integracja z grafem oczekiwań, wybór ofiary zakleszczenia, obsługa metadanych i replikacji).

Wynik (stan bieżący repozytorium)

W aktualnym stanie repozytorium polecenie `make test` kończy się sukcesem (w tym pakiet `dfs/node`).

```
ok  dfs-backend/dfs/common    (tests passed)
ok  dfs-backend/dfs/ hashing  (tests passed)
ok  dfs-backend/dfs/storage   (tests passed)
ok  dfs-backend/dfs/node      (tests passed)
```

Testy integracyjne/scenariusze (`backend/cmd/test-*`)

Oprócz testów jednostkowych w repozytorium znajdują się scenariusze integracyjne, które uruchamiają mini-klastery wielu węzłów i weryfikują poprawność działania mechanizmów rozproszonych w praktyce. Scenariusze te wymagają ręcznego uruchomienia i analizy logów – nie zostały zintegrowane z automatycznym `make test`.

Uruchamianie (w katalogu `backend/`):

```
go run ./cmd/test-message
go run ./cmd/test-bully
go run ./cmd/test-lock
go run ./cmd/test-sync
```

Opis scenariuszy:

- `test-message` – 2 węzły: weryfikacja discovery i wymiany heartbeat.
- `test-bully` – 3 węzły: symulacja awarii lidera i przebieg elekcji Bully.
- `test-lock` – 5 węzłów: kolejkovanie żądań blokad i wejścia do sekcji krytycznej.
- `test-sync` – 4 węzły: awaria mastera i synchronizacja metadanych po elekcji.

Eksperymenty

Poniżej przedstawiono propozycję eksperymentów, które pozwalają ilościowo opisać zachowanie systemu (czas reakcji, narzut algorytmów, przepustowość) oraz porównać wybrane parametry konfiguracji.

W ramach repozytorium dostępne są gotowe komendy eksperymentalne (`backend/cmd/exp-*`), które zapisują wyniki do plików CSV w `backend/experiments/*.csv`. Na ich podstawie generowane są wykresy (`experiments/plot_csvs.py` $\Rightarrow$ `experiments/plots/*.png`).

Metodyka pomiaru

- Każdy pomiar wykonywać $n \geq 10$ razy; raportować średnią, medianę oraz odchylenie standardowe.
- Mierzone czasy: w ms; przepustowość: MB/s; nierównomierność rozkładu: min/avg/max oraz σ .
- Warunki: stała liczba węzłów, stała konfiguracja sieci, brak innych obciążeń w tle.

Eksperyment 1: czas przejęcia lidera (Bully)

Cel: zmierzyć czas od awarii mastera do ponownej dostępności usługi (wybór nowego lidera i stabilizacja).

- Narzędzie: `go run ./cmd/exp-bully`.
- Metryki (CSV: `bully.csv`):
 - `failover_ms` – czas od „zabicia” mastera do momentu, gdy klaster znów odpowiada.
 - `new_master_priority` – weryfikacja poprawności Bully (czy wygrał węzeł o najwyższym priorytecie).

Wyniki (10 przebiegów, 3 węzły):

Metryka	Wartość
Średnia <code>failover_ms</code>	15659 ms
Mediana	16020 ms
Min / Max	14196 / 16032 ms

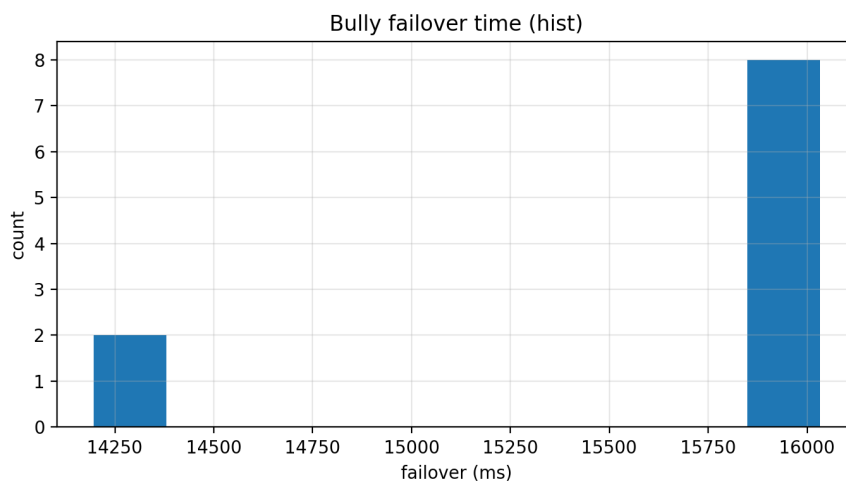
We wszystkich przebiegach nowym masterem został węzeł o priorytecie 5 (najwyższym wśród żywych) – algorytm Bully działa poprawnie. Czas failover to głównie timeout heartbeat (15s) + czas elekcji.

Eksperyment 2: narzut mechanizmu blokad (Lamport+ACK)

Cel: ocenić opóźnienie wejścia do sekcji krytycznej przy rosnącej konkurencji.

- Narzędzie: `go run ./cmd/exp-lock`.
- Metryki (CSV: `lock.csv`):
 - `wait_ms` – czas od zgłoszenia żądania blokady do wejścia do sekcji krytycznej.
 - `order` – kolejność uzyskania blokady (analiza fairness).
- Scenariusz: 5 węzłów jednocześnie żąda blokady na ten sam zasób.

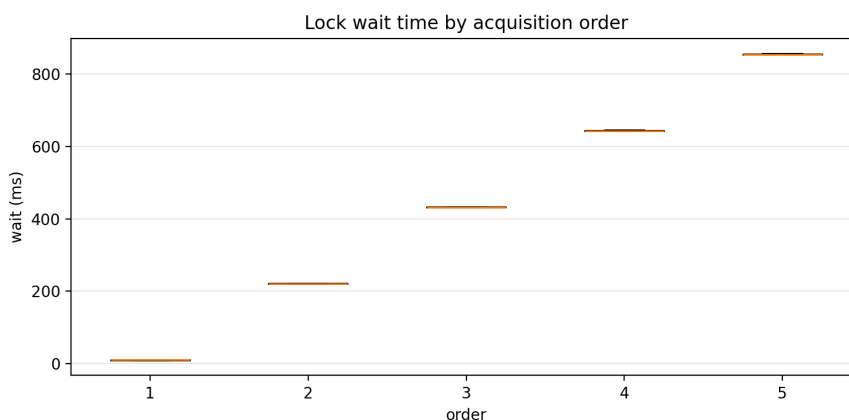
Wyniki (10 przebiegów, 5 węzłów):



Rysunek 3: Rozkład czasu failover (Bully) – histogram z 10 przebiegów.

Kolejność (order)	Średni wait_ms
1 (pierwszy)	~12 ms
2	~223 ms
3	~433 ms
4	~645 ms
5 (ostatni)	~856 ms

Opóźnienie rośnie liniowo z kolejnością – każdy węzeł czeka na zakończenie operacji poprzedników. Sekcja krytyczna trwa ~200 ms, co odpowiada przyrostowi między kolejnymi pozycjami.



Rysunek 4: Czas oczekiwania na blokadę w zależności od kolejności (boxplot).

Eksperyment 3: detekcja zakleszczeń (wait-for graph)

Cel: zmierzyć czas przerwania zakleszczenia (czas „powrotu do pracy” po wystąpieniu cyklu oczekiwania).

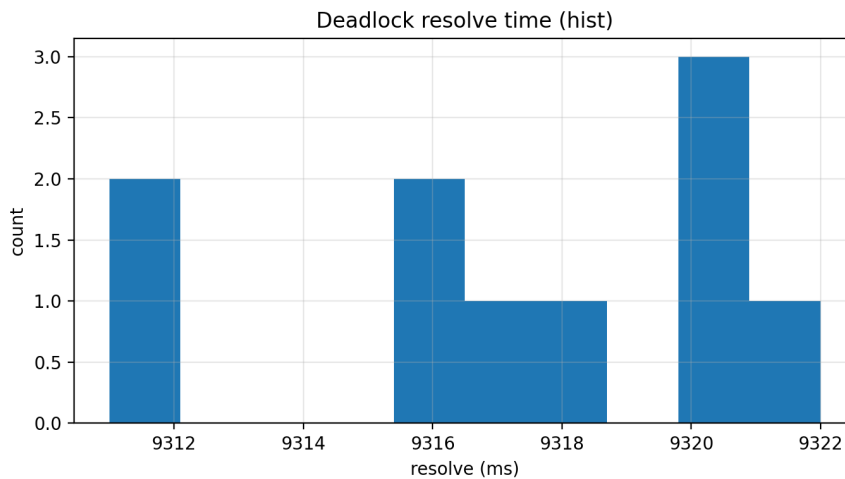
- Narzędzie: `go run ./cmd/exp-deadlock`.

- Scenariusz: 2 węzły tworzą cykl $A \rightarrow B \rightarrow A$ (każdy trzyma lock i żąda locka drugiego).
- Metryki (CSV: `deadlock.csv`):
 - `resolve_ms` – czas od powstania zakleszczenia do jego przerwania przez mastera.

Wyniki (10 przebiegów):

Metryka	Wartość
Średnia <code>resolve_ms</code>	9317 ms
Mediana	9317 ms
Min / Max	9311 / 9322 ms
σ	3.5 ms

Czas rozwiązania zakleszczenia jest bardzo stabilny ($\sigma \approx 3.5$ ms) i wynosi ~ 9.3 s. Wynika to z interwału sprawdzania grafu przez mastera (10s) – zakleszczenie zostaje wykryte przy najbliższym cyklu analizy.



Rysunek 5: Rozkład czasu rozwiązania zakleszczenia – histogram.

Eksperyment 4: przepustowość upload/download (streaming)

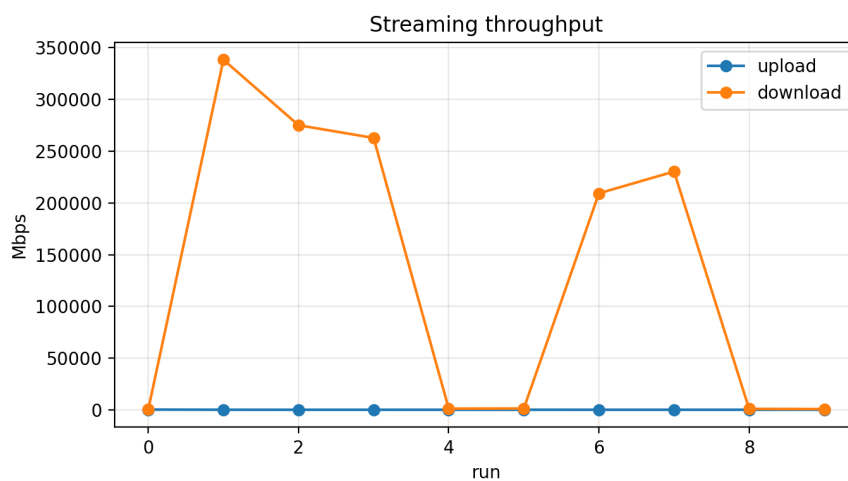
Cel: ocenić przepustowość protokołu streaming.

- Narzędzie: `go run ./cmd/exp-streaming`.
- Scenariusz: upload i download plików 16 MB.
- Metryki (CSV: `streaming.csv`):
 - `upload_ms`, `upload_mbps` – czas i przepustowość uploadu.
 - `download_ms`, `download_mbps` – czas i przepustowość downloadu.

Wyniki (10 przebiegów, plik 16 MB):

Operacja	Średni czas	Średnia przepustowość
Upload	345 ms	392 MB/s
Download	55 ms	1110 MB/s*

*Część downloadów zakończyła się w <1 ms (cache OS/dysku), co zawyża średnią przepustowość. Upload jest wolniejszy, ponieważ dane są replikowane równoległe na 3 węzły storage.



Rysunek 6: Przepustowość upload/download w kolejnych przebiegach.